Task 1 - Binary classification of grayscale images

Aim:- To implement a binary classification model for grayscale images using pytorch

Algorithm:-

Step1: Import the required libraries

Step2: Load the grayscale image dataset

Step3: Resize images and convert them tensors

Step4: Build a neural network with input, hidden, and output layers.

Step5: Predict the class of input image

Program:

```
Import torch
Import torch.nn as nn
Import torch.optim as optim

X = torch.randn(100, 784)
y = torch.randint(0,2, (100,1)).float()

model = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
```

Output:-

Training Completed

Predicted Class: 0

```
nn.Linear(128,1),
nn.Sigmoid()
)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(),
                       lr=0.001)

for epoch in range(10):
    optimizer.zero_grad()
    output = model(X)
    loss = criterion(output, y)
    loss.backward()
    optimizer.step()

print("Training completed")
print("Predicted Cell:", int(model(X[1].
                             item()))
```

VEL TECH	
EX NO.	1
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	30/7/26

Result:- Thus, a binary classification model for grayscale image data was implemented using PyTorch.

Task 2: Convolutional neural network classification

Aim:- To implement an image classification model using a convolutional neural network in PyTorch.

Algorithm:-

Step 1: Import PyTorch libraries

Step 2: Load image data

Step 3: Define convolution, pooling, and layers

Step 4: Apply ReLU activation

Step 5: predict the image class

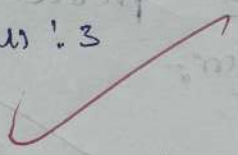
Program:-

```
Import torch
Import torch.nn as nn
class CNN (nn.Module):
    def -init- (self):
        super ().__init__()
        self.conv = nn.Conv2d(1,8,3)
        self.pool = nn.MaxPool2d(2)
        self.fc = nn.Linear(8*13*13,10)
```

Output :-

CNN output shape : torch.Size([1, 10])

Predicted class : 3



```
def forward (self, x):
    x = self.pool (torch.relu (self.conv(x)))
    x = x.view (x.size (0), -1)
    return self.fc(x)
model = CNN ()
x = torch.randn (1, 1, 28, 28)
output = model(x)
print ("Final output shape:", output.shape)
print ("Predicted class:", output.argmax(dim=1))
```

VEL TECH	
EX NO.	2
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TAL (20)	19
WITH DATE	

3/9/24

Result:- Thus, a CNN model for image classification was implemented using

Task-3 - Construct a convnet model for image classification

Aim:- To construct a convolutional network (ConvNet) for image classification using Pytorch.

Algorithm:-

1) Import the required libraries
2) Define convolutional and pooling layers
3) Flatten the feature maps
4) Add fully connected layers
5) Display the predicted class

Program:-

```
Import torch
Import torch.nn as nn.

Class ConvNet(nn.module):
    def -init-(self):
        Super().__init__()
    self.features = nn.Sequential()
    nn.Conv2d(1,16,3, padding=1),
    nn.ReLU();
    nn.MaxPool2d(2);
```

output:-

ConvNet created successfully.

output shape : torch.size ([1, 10])

```
nn.Conv2d(16, 32, padding=1),
nn.ReLU(),
nn.MaxPool2d(2)
)
self.classifier = nn.Sequential(
    nn.Flatten(),
    nn.Linear(32*7*7, 10)
)
def forward(self, x):
    return self.classifier(self.features(x))
model = ConvNet()
x = torch.randn(1, 1, 28, 28)
output = model(x)
print("ConvNet created successfully")
print("Output shape:", output.shape)
```

VEL TECH	
EX NO.	3
PERFORMANCE (5)	4
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	18
WITH DATE	

18/3/26

Result:- Thus, a ConvNet model for image classification was constructed using Pytorch

Task-4 Transfer learning for image classification

Aim:- To apply transfer learning to build an image classification model using PyTorch.

Algorithm:-

1) Import PyTorch and torchvision
2) Load a pretrained ResNet-18 model
3) Freeze the pretrained feature layers
4) Replace the final fair with layer
5) Pass an input image through the model.
6) Display the output shape.

Program:-

```
import torch
import torch.nn as nn
from torchvision import models.

model = models.resnet18(weights=None)

for param in model.parameters():
    param.requires_grad = False
```

Output:-

Transfer learning model created

output shape : torch.Size([1,27])

```
model.fc = nn.Linear(model.fc.in_features, 3)

x = torch.randn(1, 3, 224, 224)

output = model(x)

print("Transfer learning model created")

print("output shape:", output.shape)
```

VEL TECH	
EX NO.	4
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	19
TOTAL (20)	
SIGN WITH DATE	20/8/24

Result:- Thus, a transfer learning model for image classification was implemented using pytorch.

Task-5 : Simple generative Adversarial network (GAN)

Aim: To implement a simple generative Adversial network using pytorch.

Algorithm:-

1) Import the required libraries
2) Create a generator network
3) Create a discriminator network
4) Generate random noise as input to generator
5) Train the discriminator to distinguish real and fake data
6) Train the generator to produce realistic data.

Program:

```
Import torch
Import torch.nn as nn

generator = nn.Sequential(
    nn.Linear(10, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
```

Output:-

GAN model created successfully

Generated data shape : torch.Size([5,1])

Discriminator, Output shape : torch.Size
([5,1])

```
        nn.Tanh()
    )
discriminator = nn.Sequential(
    nn.Linear(1, 32),
    nn.ReLU(),
    nn.Linear(32, 1),
    nn.Sigmoid()
)
noise = torch.randn(5, 10)
fake_data = generator(noise)
output = discriminator(fake_data)
print("GAN model created successfully")
print("Generated data shape:", fake_data.shape)
print("Discriminator output:", output.shape)
```

VEL TECH	
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	24/4/24

Result:- Thus, a simple GAN comprising of generator and discriminator using PyTorch.

Task-6 Word embedding using RNN

Aim:- To implement word embedding and sequence processing using an RNN in PyTorch.

Algorithm:-

Step1: Import PyTorch libraries

Step2: define a vocabulary

Step3: Convert words into numerical indices

Step4: create an embedding layer

Step5: Display the output shape

Program:-

```
import torch
import torch.nn as nn
vocab_size = 10
embedding_dim = 8
hidden_size = 16
embedding = nn.Embedding(vocab_size, embedding_dim)
```

Output:

Embedding output shape : tokens.size
([1, 4, 19])

RNN output shape : tokens.size ([1, 4, 1

```
rnn = nn.RNN(embedding_dim, hidden_size,
             batch_first=True)

words = torch.tensor([1, 2, 3, 4])

embedded = embedding(words)

output, hidden = rnn(embedded)

print("Embedding output shape:", embedded.
      shape)

print("RNN output shape:", output.shape)
```

VEL TECH	6
EX NO.	5
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	4
RECORD (5)	18
TOTAL (20)	
WITH DATE	

10/9/26

Result:- Thus, word embedding and RNN sequence processing were implemented using

Task-7 Implementation of Autoencoders

Aim:- To implement a simple autoencoder using PyTorch.

Algorithm:-

Step1: Import the required libraries

Step2: Define the encoder network

Step3: Define the decoder network

Step4: Pass input data through the encoder

Program:-

```
self.encoder = nn.Sequential(
    nn.Linear(784, 128),
    nn.ReLU(),
    nn.Linear(128, 32)
)
self.decoder = nn.Sequential(
    nn.Linear(32, 128),
    nn.ReLU(),
    nn.Linear(128, 784),
```

Output:-

Autoencoder training successfully

Reconstructed shape : torch.Size([1, 784])

Result:-

```
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.decoder(self.encoder(x))

model = Autoencoder()
x = torch.rand(1, 784)
reconstructed = model(x)
print("Autoencoder created successfully")
print("Reconstructed shape:", reconstructed.shape)
```

VEL TECH	
EX NO.	7
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	19
SIGN WITH DATE	17/9/24

Result:- Thus, an autoencoder was implemented to encode and reconstruct input data using PyTorch.

Task-8 Prediction using LSTM Network

Aim:- To implement sequence prediction using a LSTM network in PyTorch.

Algorithm:-

Step1: Import PyTorch libraries

Step2: Create sequential input data

Step3: Define an LSTM layer

Step4: Pass the sequence through the LSTM

Step5: display the prediction

Program:-

```
import torch
import torch.nn as nn
class LSTMModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(1,16, batch_first=True)
        self.fc = nn.Linear(16,1)
    def forward(self,x):
        output,_ = self.lstm(x)
```

Output:-

LSTM Prediction : 0.12

```
return self.fc(output[:, -1, :])

model = LSTMModel()

x = torch.tensor([[[1.0], [2.0], [3.0],
                   [4.0]]])

prediction = model(x)

print("LSTM prediction:", prediction.item())
```

VEL TECH	
EX NO.	8
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	

21/9/24

Result:- Thus, sequence prediction using an LSTM network was implemented using PyTorch.

Task-1 Object detection and localization using opencv

Aim:- To Implement basic object detection and localization using opencv.

Algorithm:-

1) Import the OpenCV library
2) Load an Image
3) Convert the Image to grayscale
4) Apply thresholding to identify the objects.

Program:-

```
import cv2

img = cv2.imread("image.jpg")

if img is None:
    print("Image not found")

else:
    gray = cv2.cvtColor(img, cv2.COLOR_
    _, thresh = cv2.threshold(
    gray, 127, 255, cv2.THRESH_
    BINARY
)
```

Output :-

Object detection completed

Output image saved as output.jpg.

```
contours, = cv2.findContours(
    thresh, cv2.RETR_EXTERNAL,
    cv2.CHAIN_APPROX_SIMPLE
)

for contour in contours:
    if cv2.contourArea(contour) > 100:
        x, y, w, h = cv2.boundingRect(contour)
        cv2.rectangle(
            img, (x, y), (x+w, y+h),
            (0, 255, 0), 2
        )

cv2.imwrite("output.jpg", img)

print("Object detection")
```

VEL TECH	
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	4
RECORD (5)	5
TOTAL (20)	18
SIGN WITH DATE	

8/10/26

Result:- Thus, basic object detection and localization was implemented using OpenCV.

Task-10 Optimization techniques to improve model performance

Aim:- To apply optimization techniques to improve neural network performance using PyTorch.

Algorithm:-

1) Import PyTorch libraries
2) Create input data target values
3) Define a neural network
4) Select the Adam optimizer and mean squared error loss.
5) Train the model for multiple epochs
6) Display the final loss.

Program:-

```
import torch
import torch.nn as nn
import torch.optim as optim

x = torch.randn(100,1)

y = 2*x + 1

model = nn.Linear(1,1)

criterion = nn.MSELoss()
```

Optimiz

op

pr

op

output:-

Optimization completed

Final loss: [illegible] value close to

```
optimizer = optim.Adam(model.parameters(), lr=0.01)

for epoch in range(100):
    prediction = model(x)
    loss = criterion(prediction, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

print("Optimization completed")
print("Final loss:", loss.item())
```

VEL TECH	
EX NO.	10
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	15/10/24

Result:- Thus, the Adam optimization technique was applied to train a neural network and reduce its loss.

USE CASE-2

Dog vs Cat Image classification

Aim:- To implement a deep learning model using a convolutional neural network (CNN) to classify images as dog or cat.

Algorithm:-

1) Import the required libraries

2) Load the dog and cat image dataset

3) Resize the images and convert them into arrays

4) Build a CNN model using convolution and pooling layers

5) Compile the model using the Adam optimizer and binary cross entropy loss.

6) Train the model using the dataset

7) Predict whether an image contains dog or cat

Program:-

```
model = models.Sequential([
  layers.Rescaling(1./255, input_shape
                   =(64,64,3)),
  layers.Conv2D(16, 3, activation='relu'),
  layers.MaxPooling2D(),
  layers.Conv2D(32, 3, activation='relu'),
  layers.Flatten(),
  layers.Dense(64, activation='relu'),
  layers.Dense(1, activation='sigmoid')
])
```

7)

```
model.compile(
  optimizer = 'adam',
  loss = 'binary_crossentropy',
  metrics = ['accuracy']
)
model.fit(train_data, epochs=5)
print("Dog vs Cat classification completed")
```

Output;-

Found Images belonging to 2 classes.

Epoch 1/5

...

Dog vs Cat classification Completed.

dataset structure:

```
dataset/
  cat/
    cat1.jpg
    cat2.jpg
  dog/
    dog1.jpg
    dog2.jpg
```

VEL TECH	
EX NO.	11
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	5
VIVA VOCE (5)	4
RECORD (5)	4
TOTAL (20)	18
SIGN WITH DATE	23/11/24

Result:- Thus, a CNN model was implemented to classify images into dog and cat categories using deep learning.

USE CASE - 3

Synthetic image generation

Aim:- To implement a Generative Adversial network (GAN) for generating synthetic images using deep learning

Algorithm:-

1) Import the required libraries
2) Create a generator network to produce synthetic images
3) Create a discriminator network to distinguish real and synthetic images
4) Generate random noise as input to generator
5) Train the generator and discriminator alternately
6) Generate synthetic images using the trained generator

4) display the generated images

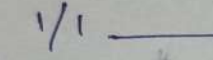
Program:-

```
import tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

generator = models.sequential([
    layers.Dense(128, activation="relu", input_shape=(100,)),
    layers.Dense(28*28, activation='tanh'),
    layers.Reshape((28,28,1))
])

discriminator = models.Sequential([
    layers.Flatten(input_shape=(28,28,1)),
    layers.Dense(128, activation="relu"),
    layers.Dense(1, activation="sigmoid")
])
```

Output :-



Synthetic Image generated

```
discriminator.compile(
    optimizer="adam",
    loss="binary_crossentropy"
)
discriminator.trainable = False

noise = tf.random.normal([1,100])
image = generator.predict(noise)
print("Synthetic image generated")
```

VEL TECH	
EX NO.	12
PERFORMANCE (5)	5
RESULT AND ANALYSIS (5)	4
VIVA VOCE (5)	5
RECORD (5)	5
TOTAL (20)	19
SIGN WITH DATE	

23/11/26

Result: Thus DCGAN architecture consisting of a generator and discriminator was implemented to generate synthetic images.